Link Model with Extra Fields

In the previous example we never interacted directly with the `HeroTeamLink` model, it was all through the automatic **many-to-many** relationship.

But what if we needed to have additional data to describe the link between the two models?

Let's say that we want to have an extra field/column to say if a hero **is still training** in that team or if they are already going on missions and stuff.

Let's see how to achieve that.

Link Model with Two One-to-Many

The way to handle this is to explicitly use the link model, to be able to get and modify its data (apart from the foreign keys pointing to the two models for `Hero` and `Team`).

In the end, the way it works is just like two **one-to-many** relationships combined.

A row in the table `heroteamlink` points to **one** particular hero, but a single hero can be connected to **many** hero-team links, so it's **one-to-many**.

And also, the same row in the table `heroteamlink` points to **one** team, but a single team can be connected to **many** hero-team links, so it's also **one-to-many**.

> **Tip**
>
> The previous many-to-many relationship was also just two one-to-many relationships combined, but now it's going to be much more explicit.

Update Link Model

Let's update the `HeroTeamLink` model.

We will add a new field `is_training`.

And we will also add two **relationship attributes**, for the linked `team` and `hero`:

Python 3.10+

```python
# Code above omitted 👆

class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

```
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

The new **relationship attributes** have their own `back_populates` pointing to new relationship attributes we will create in the `Hero` and `Team` models:

- `team`: has `back_populates="hero_links"`, because in the `Team` model, the attribute will contain the links to the **team's heroes**.
- `hero`: has `back_populates="team_links"`, because in the `Hero` model, the attribute will contain the links to the **hero's teams**.

Info

In SQLAlchemy this is called an Association Object or Association Model.

I'm calling it **Link Model** just because that's easier to write avoiding typos. But you are also free to call it however you want. 😉

Update Team Model

Now let's update the `Team` model.

We no longer have the `heroes` relationship attribute, and instead we have the new `hero_links` attribute:

Python 3.10+

```python
# Code above omitted 👆

class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

```python
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Update Hero Model

The same with the `Hero` model.

We change the `teams` relationship attribute for `team_links`:

Python 3.10+

```python
# Code above omitted 👆

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

```
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Create Relationships

Now the process to create relationships is very similar.

But now we create the **explicit link models** manually, pointing to their hero and team instances, and specifying the additional link data (`is_training`):

Python 3.10+

```
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
```

```
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

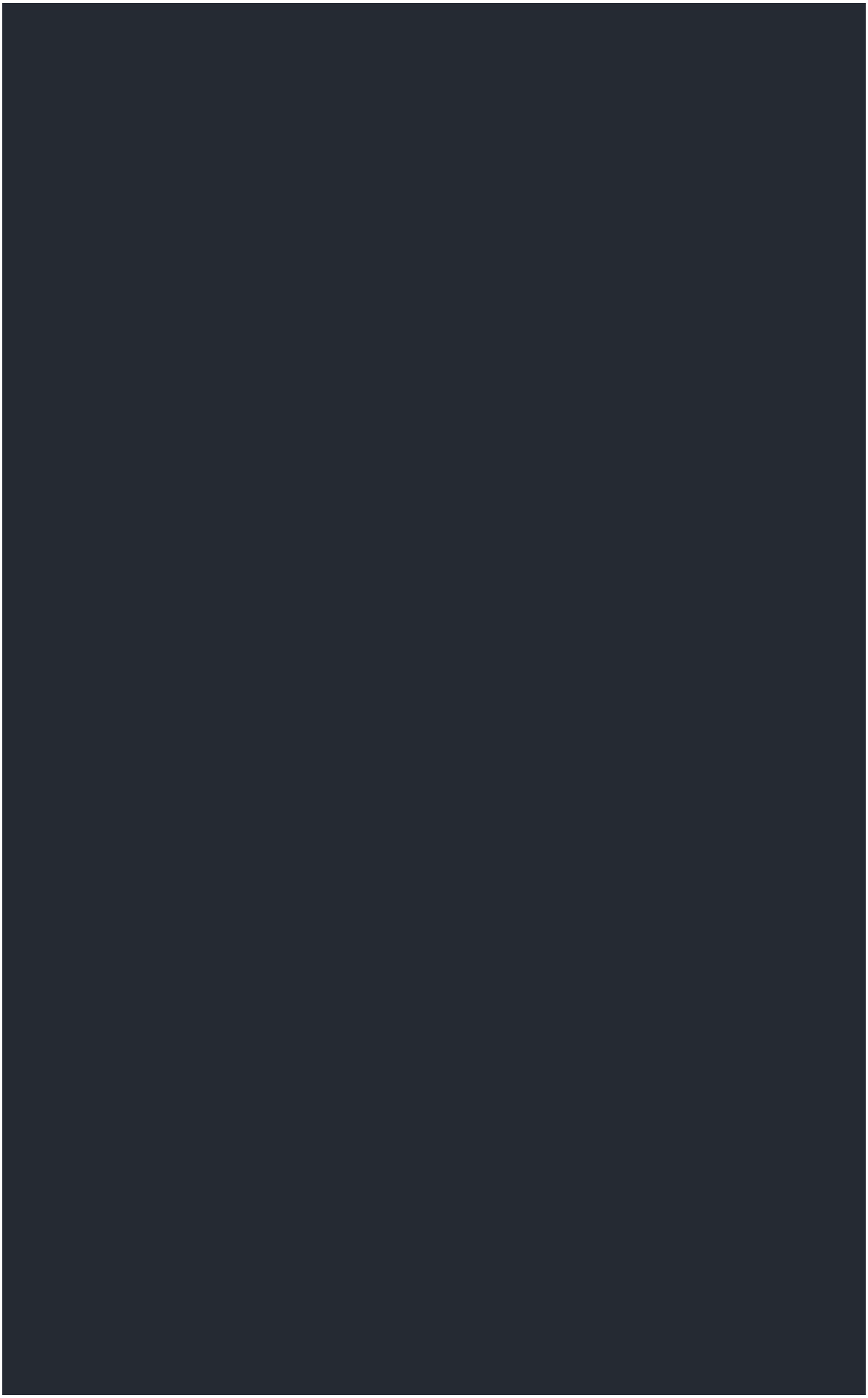
```
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

We are just adding the link model instances to the session, because the link model instances are connected to the heroes and teams, they will be also automatically included in the session when we commit.

Run the Program

Now, if we run the program, it will show almost the same output as before, because it is generating almost the same SQL, but this time including the new `is_training` column:

```bash
```



Add Relationships

Now, to add a new relationship, we have to create a new `HeroTeamLink` instance pointing to the hero and the team, add it to the session, and commit it.

Here we do that in the `update_heroes()` function:

Python 3.10+

```
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

```python
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Run the Program with the New Relationship

If we run that program, we will see the output:

```bash
```

Update Relationships with Links

Now let's say that **Spider-Boy** has been training enough in the **Preventers**, and they say he can join the team full time.

So now we want to update the status of `is_training` to `False`.

We can do that by iterating on the links:

Python 3.10+

```
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:

# Code here omitted 👈

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)
    is_training: bool = False

    team: "Team" = Relationship(back_populates="hero_links")
    hero: "Hero" = Relationship(back_populates="team_links")


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    hero_links: list[HeroTeamLink] = Relationship(back_populates="team")


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_links: list[HeroTeamLink] = Relationship(back_populates="hero")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
        )
        hero_spider_boy = Hero(
            name="Spider-Boy",
            secret_name="Pedro Parqueador",
        )
        deadpond_team_z_link = HeroTeamLink(team=team_z_force, hero=hero_deadpond)
        deadpond_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_deadpond, is_training=True
        )
        spider_boy_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_spider_boy, is_training=True
        )
        rusty_man_preventers_link = HeroTeamLink(
            team=team_preventers, hero=hero_rusty_man
        )

        session.add(deadpond_team_z_link)
        session.add(deadpond_preventers_link)
```

```python
        session.add(spider_boy_preventers_link)
        session.add(rusty_man_preventers_link)
        session.commit()

        for link in team_z_force.hero_links:
            print("Z-Force hero:", link.hero, "is training:", link.is_training)

        for link in team_preventers.hero_links:
            print("Preventers hero:", link.hero, "is training:", link.is_training)


def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        spider_boy_z_force_link = HeroTeamLink(
            team=team_z_force, hero=hero_spider_boy, is_training=True
        )
        team_z_force.hero_links.append(spider_boy_z_force_link)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.team_links)
        print("Z-Force heroes:", team_z_force.hero_links)

        for link in hero_spider_boy.team_links:
            if link.team.name == "Preventers":
                link.is_training = False

        session.add(hero_spider_boy)
        session.commit()

        for link in hero_spider_boy.team_links:
            print("Spider-Boy team:", link.team, "is training:", link.is_training)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Run the Program with the Updated Relationships

And if we run the program now, it will output:

```bash
```

Recap

If you need to store more information about a **many-to-many** relationship you can use an explicit link model with extra data in it. 🤓